

✓ Welcome to Colab!

```
def check_win(board, player):
    """Returns True if the specified player has won the game."""
    win_conditions = [
        (0, 1, 2), (3, 4, 5), (6, 7, 8), # Rows
        (0, 3, 6), (1, 4, 7), (2, 5, 8), # Columns
        (0, 4, 8), (2, 4, 6)             # Diagonals
    ]
    return any(board[a] == board[b] == board[c] == player for a, b, c in win_conditions)

def check_tie(board):
    """Returns True if the board is completely full."""
    return " " not in board

def print_inline_boards(boards_list):
    """Prints a list of boards horizontally side-by-side with step markers."""
    for i in range(3):
        row_str = ""
        for step, board in enumerate(boards_list):
            idx = i * 3
            # Format one line of a 3x3 grid
            grid_line = f" {board[idx]} | {board[idx+1]} | {board[idx+2]} "

            # Add step text label on the middle row
            if i == 1:
                label = f" (Step {step}) → " if step < len(boards_list) - 1 else " (Final) "
            else:
                label = " " if step < len(boards_list) - 1 else " "

            row_str += grid_line + label
        print(row_str)
        if i < 2:
            divider_str = ""
            for step in range(len(boards_list)):
                divider_str += "---|---|---" + (" " if step < len(boards_list) - 1 else "")
            print(divider_str)
        print("\n" + "="*80 + "\n")

def explore_tree(board, current_player, history=None):
    """Recursively traverses the tree and prints each path visually as it completes."""
    if history is None:
        # Start history with a copy of the root half-played board state
        history = [list(board)]

    last_player = "O" if current_player == "X" else "X"

    # Base Case 1: Last player won
    if check_win(board, last_player):
        global path_counter
        path_counter += 1
        print(f"PATH #{path_counter} | Outcome: {last_player} Wins | Path Cost: {len(history) - 1} moves")
        print_inline_boards(history)
        return

    # Base Case 2: Tie game
    if check_tie(board):
        path_counter += 1
        print(f"PATH #{path_counter} | Outcome: Tie Game | Path Cost: {len(history) - 1} moves")
        print_inline_boards(history)
        return

    # Find available spots
    empty_cells = [i for i, cell in enumerate(board) if cell == " "]

    # Branch out to all possible next turns
    for cell in empty_cells:
        next_board = list(board)
        next_board[cell] = current_player

        next_player = "O" if current_player == "X" else "X"
        explore_tree(next_board, next_player, history + [next_board])
```

```
# --- EXECUTION ---
if __name__ == "__main__":
    # Global tracking counter for paths
    path_counter = 0

    # Define your half-played board structure
    # X | O | X
    # ---|---|---
    # O | X |
    # ---|---|---
    # | |
    initial_board = [
        "O", " ", "X",
        "X", " ", " ",
        "X", "O", "O "
    ]

    next_up = "O"

    print("=" * 80)
    print("STARTING HALF-PLAYED BOARD STATE:")
    print(f" {initial_board[0]} | {initial_board[1]} | {initial_board[2]} ")
    print("---|---|---")
    print(f" {initial_board[3]} | {initial_board[4]} | {initial_board[5]} ")
    print("---|---|---")
    print(f" {initial_board[6]} | {initial_board[7]} | {initial_board[8]} ")
    print(f"\nNext turn: Player {next_up}")
    print("=" * 80 + "\n")

    explore_tree(initial_board, next_up)
```

Next turn: Player O

=====

PATH #1 | Outcome: Tie Game | Path Cost: 3 moves

```

O |   | X          O | O | X          O | O | X          O | O | X
---|---|---      ---|---|---      ---|---|---      ---|---|---
X |   |           X |   |           X | X |           X | X | O   (Final)
---|---|---      ---|---|---      ---|---|---      ---|---|---
X | O | O         X | O | O         X | O | O         X | O | O
```

=====

PATH #2 | Outcome: Tie Game | Path Cost: 3 moves

```

O |   | X          O | O | X          O | O | X          O | O | X
---|---|---      ---|---|---      ---|---|---      ---|---|---
X |   |           X |   |           X |   | X       (Step 2) → X | O | X   (Final)
---|---|---      ---|---|---      ---|---|---      ---|---|---
X | O | O         X | O | O         X | O | O         X | O | O
```

=====

PATH #3 | Outcome: Tie Game | Path Cost: 3 moves

```

O |   | X          O |   | X          O | X | X          O | X | X
---|---|---      ---|---|---      ---|---|---      ---|---|---
X |   |           X | O |           X | O |           X | O | O   (Final)
---|---|---      ---|---|---      ---|---|---      ---|---|---
X | O | O         X | O | O         X | O | O         X | O | O
```

=====

PATH #4 | Outcome: Tie Game | Path Cost: 3 moves

```

O |   | X          O |   | X          O |   | X          O | O | X
---|---|---      ---|---|---      ---|---|---      ---|---|---
X |   |           X | O |           X | O | X       (Step 2) → X | O | X   (Final)
---|---|---      ---|---|---      ---|---|---      ---|---|---
X | O | O         X | O | O         X | O | O         X | O | O
```

=====

PATH #5 | Outcome: Tie Game | Path Cost: 3 moves

```

O |   | X          O |   | X          O | X | X          O | X | X
---|---|---      ---|---|---      ---|---|---      ---|---|---
X |   |           X |   | O       (Step 1) → X |   | O   (Step 2) → X | O | O   (Final)
---|---|---      ---|---|---      ---|---|---      ---|---|---
```

```

---|---|--- (Step 0) → ---|---|--- (Step 1) → ---|---|--- (Step 2) → ---|---|--- (Final)
X |   |   X |   | 0 X | X | 0 X | X | 0
---|---|--- ---|---|--- ---|---|--- ---|---|---
X | 0 | 0 X | 0 | 0 X | 0 | 0 X | 0 | 0
=====

```

```

import random
import time

class VacuumEnvironment:
    def __init__(self):
        # Locations A and B can be either 'Clean' or 'Dirty'
        self.locations = {
            "A": random.choice(["Dirty"]),
            "B": random.choice(["Dirty"])
        }
        # Agent starts in a random location
        self.agent_location = random.choice(["A"])
        self.performance_score = 0

    def get_percept(self):
        """Returns the current location and its condition."""
        return self.agent_location, self.locations[self.agent_location]

    def execute_action(self, action):
        """Changes the environment state based on the agent's action."""
        print(f"👤 Agent decides to: {action}")

        if action == "Suck":
            if self.locations[self.agent_location] == "Dirty":
                self.locations[self.agent_location] = "Clean"
                self.performance_score += 10 # Reward for cleaning
                print(f"🧹 Sucked dirt in Location {self.agent_location}.")
            else:
                self.performance_score -= 1 # Penalty for wasting energy
                print(f"⚠️ Tried to suck, but Location {self.agent_location} was already clean.")

        elif action == "Move_Right":
            if self.agent_location == "A":
                self.agent_location = "B"
                print(f"➡️ Moved from A to B.")
                self.performance_score -= 1 # Energy cost for movement

        elif action == "Move_Left":
            if self.agent_location == "B":
                self.agent_location = "A"
                print(f"⬅️ Moved from B to A.")
                self.performance_score -= 1 # Energy cost for movement

    def display_status(self):
        """Prints the current state of the environment."""
        print("\n" + "="*40)
        print(f"📍 Agent Location: {self.agent_location}")
        print(f"🏠 Status - Room A: {self.locations['A']} | Room B: {self.locations['B']}")

    def is_all_clean(self):
        """Checks if both locations are clean."""
        return self.locations["A"] == "Clean" and self.locations["B"] == "Clean"

class ReflexVacuumAgent:
    def __init__(self):
        pass

    def program(self, percept):
        """Reflex Agent rule logic: Condition -> Action"""
        location, status = percept

        if status == "Dirty":
            return "Suck"
        elif location == "A":
            return "Move_Right"
        elif location == "B":
            return "Move_Left"

```

```

# --- EXECUTION LOOP ---
if __name__ == "__main__":
    # Initialize the world and the agent
    env = VacuumEnvironment()
    agent = ReflexVacuumAgent()

    print("🚀 Initializing Two-Location Vacuum Environment...")
    env.display_status()
    time.sleep(1)

    step = 1
    # Run the simulation until both rooms are clean
    while not env.is_all_clean() and step <= 5:
        print(f"\n🕒 --- STEP {step} ---")

        # 1. Agent senses the environment (Percept)
        percept = env.get_percept()
        print(f"👁️ Agent perceives: Location {percept[0]} is {percept[1]}")

        # 2. Agent chooses an action based on rules
        action = agent.program(percept)

        # 3. Environment reacts to the action
        env.execute_action(action)

        # 4. Show updated status
        env.display_status()

        step += 1
        time.sleep(1.5)

    if env.is_all_clean():
        print("\n🌟 Success! All locations are perfectly clean. 🌟")
    else:
        print("\n🛑 Simulation ended due to maximum steps reached.")

```

🚀 Initializing Two-Location Vacuum Environment...

```

=====
📍 Agent Location: A
🏠 Status - Room A: Dirty | Room B: Dirty

🕒 --- STEP 1 ---
👁️ Agent perceives: Location A is Dirty
📖 Agent decides to: Suck
🧹 Sucked dirt in Location A.

=====
📍 Agent Location: A
🏠 Status - Room A: Clean | Room B: Dirty

🕒 --- STEP 2 ---
👁️ Agent perceives: Location A is Clean
📖 Agent decides to: Move_Right
➡️ Moved from A to B.

=====
📍 Agent Location: B
🏠 Status - Room A: Clean | Room B: Dirty

🕒 --- STEP 3 ---
👁️ Agent perceives: Location B is Dirty
📖 Agent decides to: Suck
🧹 Sucked dirt in Location B.

=====
📍 Agent Location: B
🏠 Status - Room A: Clean | Room B: Clean

🌟 Success! All locations are perfectly clean. 🌟

```

